



ADS05：二项队列

来源：/home/vincent/Code/ads/Materials/Student Slides/ADS05BinomialQ_Stu.ppt

课件范围：S1-S3、S4-S6、S7-S11、S12-S13、S14-S15；共 15 张幻灯片。

正文按原课件顺序整理；S 编号指幻灯片顺序。

本讲路线图

- 二项树的递归结构 → 用二进制表示森林 → 查最小元、合并、插入与删除 → 孩子—兄弟实现 → 连续插入的线性总时间。

1. 二项树与二项队列的结构 [S1-S3]

二项队列 (Binomial Queue) 由一组满足堆序的二项树组成。用不同阶的树表示不同的二进制位，可以把队列合并转化为逐位处理与进位。[S1-S3]

1.1 二项树的递归定义与规模 [S2]

重要性：A · 必须掌握

二项树 (Binomial Tree) B_k 的高度为 k ，根的深度为 0。按阶数分类：

1. $k = 0$ ： B_0 只有一个节点。
2. $k \geq 1$ ：把一棵 B_{k-1} 的根接到另一棵 B_{k-1} 的根下面，得到 B_k 。
3. 根的孩子： B_k 的根有 k 个孩子，它们分别是一棵 $B_0, B_1, \dots, B_{k-1}$ 的根。
4. 节点数： B_k 恰好有 2^k 个节点；深度 d 的节点数是二项系数 $\binom{k}{d}$ ，其中 $0 \leq d \leq k$ 。[S2]

核心原理

每次把两棵同阶树连接，节点总数加倍；被挂接的那一半整体深度加一，因此每层计数满足 Pascal 递推。

1. 设节点总数为 n_k 。 $n_0 = 1$ ，递归构造给出 $n_k = 2n_{k-1}$ ，故 $n_k = 2^k$ 。
2. 设 $a_{k,d}$ 是 B_k 深度 d 的节点数。保留根的那棵旧树贡献 $a_{k-1,d}$ ，挂接的旧树贡献 $a_{k-1,d-1}$ 。
3. 因而 $a_{k,d} = a_{k-1,d} + a_{k-1,d-1}$ ；以 $a_{0,0} = 1$ 、越界项为 0 为边界，得到 $a_{k,d} = \binom{k}{d}$ 。[S2]

补充推导：上述递推补全了 S2 的结构观察。原生文字提取丢失了指数和组合符号，已针对该页核实 2^k 与 $\binom{k}{d}$ ，不能把 $2k$ 当作乘法。

1.2 森林与二进制表示 [S3]

重要性：A · 必须掌握

一个合法的最小二项队列同时满足三个条件：

1. **树形条件**：每棵树都是某个 B_k 。
2. **堆序条件 (Heap Order)**：父节点键不大于孩子键；每棵树的最小键在根上。
3. **阶数唯一**：每个阶数最多出现一棵树；不要求整个队列只有一棵树。[S3]

若第 k 阶树存在就记 $b_k = 1$ ，否则为 0，队列的节点数为

$$N = \sum_{k \geq 0} b_k 2^k, \quad b_k \in \{0, 1\}.$$

例如 $13 = 2^0 + 2^2 + 2^3 = (1101)_2$ ，因此森林中恰有 B_0, B_2, B_3 ，没有 B_1 。[S3]

解释补充：“唯一表示”指给定 N 后存在的树阶数组合唯一。它不意味着节点键在各棵树中的分布或同键节点的排列唯一。

2. 二项队列的基本操作 [S4–S6]

合并是二项队列的中心操作。插入是与单节点队列合并，删除最小元则把某棵树拆开后再合并。[S4–S6]

2.1 查找最小元与合并 [S4]

重要性：A · 必须掌握

查找最小元 (FindMin) 按是否缓存分类：

1. **没有最小根缓存**：比较所有非空树根。每棵树内部都满足堆序，根的最小值就是全局最小值；根的个数为 $O(\log N)$ ，所以时间为 $O(\log N)$ 。
2. **维护最小根缓存**：直接读取缓存可达到 $O(1)$ ；插入、合并或删除改变最小根时，必须相应更新缓存。[S4]

解释补充：对 $N \geq 1$ ，可用的阶数从 0 到 $\lfloor \log_2 N \rfloor$ ，根数的精确上界为 $\lfloor \log_2 N \rfloor + 1$ 。用位宽理解边界，比直接套用一个没有处理 $N = 1$ 的取整表达式更稳妥。

合并 (Merge) 按当前阶的树数分类：

1. **没有树**：本阶结果为空。
2. **只有一棵**：直接保留这棵树。
3. **有两棵同阶树**：连接为一棵下一阶树，作为进位。
4. **有三棵同阶树**：本阶保留一棵，把另外两棵连接后向下一阶进位。[S4、S10]

核心原理

同阶树连接后恰好升一阶；从低阶到高阶处理，把“每阶至多一棵树”的约束维持为二进制加法的结果位与进位。

两个 B_k 各有 2^k 个节点，连接后成为含 2^{k+1} 个节点的 B_{k+1} 。比较根键后，将较大根接到较小根下面，堆序也得到保留。每阶最多处理三棵树，连接代价为常数；总阶数为 $O(\log(N_1 + N_2))$ ，所以合并为对数时间。[S4、S9–S10]

S4 的规模示例对应 $(110)_2 + (111)_2 = (1101)_2$ ，即 $6 + 7 = 13$ ：第 1 阶的两棵树产生进位，第 2 阶加上进位后有三棵树，保留本阶一棵并继续进位，最终得到 B_0, B_2, B_3 。[S4]

2.2 插入与连续进位 [S5]

重要性：A · 必须掌握

插入（Insert）先构造一棵单节点 B_0 。从第 0 阶开始：

1. **当前阶为空**：把携带的树放入该位置，插入结束。
2. **当前阶已有树**：把两棵同阶树连接，清空原位置，把新树带到下一阶。
3. **首次空阶是 i** ：需要连接第 0 至 $i - 1$ 阶的树，再在第 i 阶停下；代价与 $i + 1$ 成正比。[S5]

这与二进制整数加一相同：低位连续的 1 变成 0，第一个 0 变成 1。从空队列依次插入 1 至 7，森林阶数变化如下；这里列的是树阶数，不是整棵树中键的具体排列。[S5]

插入后的元素个数	二进制规模	存在的树
1	1	B_0
2	10	B_1
3	11	B_1, B_0
4	100	B_2
5	101	B_2, B_0
6	110	B_2, B_1
7	111	B_2, B_1, B_0

一次插入可能沿所有已有阶数进位，最坏为 $O(\log N)$ ；从空队列开始连续插入 N 次的总时间为 $O(N)$ ，所以每次摊还为 $O(1)$ 。证明见原顺序中的 S12–S13，下文第 4 节。[S5、S12–S13]

解释补充：常数摊还界对应“沿进位链处理，并在首个空阶停止”的插入实现。若每次都调用一个无论是否还需进位都扫描全部阶数的通用合并程序，不能直接沿用这个插入代价证明。[S5、S10]

2.3 删除最小元的四步分解 [S6]

重要性：A · 必须掌握

设最小根位于 B_k ，删除最小元（DeleteMin）按以下顺序执行：

1. **找最小根**：扫描树根，确定 B_k ，用时 $O(\log N)$ 。
2. **移除整棵 B_k** ：从森林中清空对应位置，剩余森林记为 H' ；已知位置后这一步为 $O(1)$ 。
3. **移除 B_k 的根**：根的孩子组成 $B_0, \dots, B_{k-1}$ 的森林，记为 H'' ；组织这些子树需要 $O(k) \subseteq O(\log N)$ 。
4. **合并剩余两部分**：返回 $\text{Merge}(H', H'')$ ，用时 $O(\log N)$ 。[S6]

每棵子树都继承了原堆序，且 H'' 内部各阶唯一。合并恢复整个队列的各阶唯一性，最终只少一个节点。因此删除最小元总时间为 $O(\log N)$ 。[S6]

3. 数组与孩子—兄弟链的实现 [S7–S11]

队列用数组按阶索引树根，单棵树用孩子—兄弟指针表示。这样的布局使合并能直接访问同阶树，删除根后也能依次取出所有孩子子树。[S7–S8]

3.1 数据结构与孩子排列 [S7–S8]

重要性：B · 重点理解

1. **节点 BinNode**：保存键 Element、第一个孩子 LeftChild 和下一个兄弟 NextSibling。二项树可以有多个孩子，这两个指针是表示方法，并不把原树变成普通二叉堆。
2. **队列 Collection**：CurrentSize 是整个森林的节点总数；TheTrees[k] 保存 B_k 的根指针，空指针表示该阶不存在。
3. **孩子顺序**：按 S9 的头插法构造时， B_k 根的孩子链从 B_{k-1} 到 B_0 按阶数递减排列；S11 因而从索引 $k-1$ 向 0 填回子树。[S7–S11]

3.2 同阶树连接 CombineTrees [S9]

重要性：A · 必须掌握

连接前提是 T1 与 T2 阶数相同。操作分为：

1. 若 T1 根键较大，交换两棵树的角色，保证较小根位于 T1。
2. 令 $T2 \rightarrow \text{NextSibling} = T1 \rightarrow \text{LeftChild}$ ，把 T2 接到原孩子链前面。
3. 令 $T1 \rightarrow \text{LeftChild} = T2$ ，返回 T1。[S9]

只比较两个根并修改两个指针，时间为 $O(1)$ 。若输入都是 B_k ，新插入的孩子是 B_k ，后面的孩子依次为 $B_{k-1}, \dots, B_0$ ，所以输出是 B_{k+1} ，并保持孩子链按阶数递减。[S9]

3.3 合并的八种状态 [S10]

重要性：A · 必须掌握

当前阶的三项分别是 T1、T2 和从前一阶来的 Carry。表达式 $4*!!\text{Carry} + 2*!!\text{T2} + !!\text{T1}$ 把是否非空编码为三位数；位顺序是 (Carry, T2, T1)。[S10]

状态	当前非空树	本阶留在 H1 的结果	下一阶进位
000	无	空	无
001	T1	T1	无
010	T2	转移 T2，清空 H2 本阶	无
100	Carry	放入 Carry	清空旧进位
011	T1, T2	清空本阶	连接 T1, T2
101	T1, Carry	清空本阶	连接 T1, Carry
110	T2, Carry	本阶为空，清空 H2 本阶	连接 T2, Carry
111	三棵都有	保留旧 Carry	连接 T1, T2

不变量是：已处理的每个阶在结果中最多留一棵树，Carry 只携带一棵下一阶树；转移或连接后的旧数组位置不再保留相同所有权。[S10]

循环中的 i 表示阶数， j 从 1 每次乘 2，表示对应树的规模 2^i 。合并前先检查总规模不超过容量，再将总节点数累加到 $H1$ 。[S10]

解释补充：该合并会转移或连接 $H2$ 的树，属于会消耗输入结构的操作。S10 未展示 $H2 \rightarrow \text{CurrentSize}$ 归零及空队列对象的释放；实际实现必须补齐相应生命周期管理，不能把调用后的 $H2$ 当作内容仍完整的旧队列。

3.4 删除实现中的计数与断链 [S11]

重要性：B · 重点理解

1. **空队列：**先报错并返回约定的错误值，不能访问一个未确定的最小根索引。
2. **定位树根：**扫描非空根，记录最小键和阶数 $\text{MinTree}=k$ 。
3. **摘除并释放根：**清空 $\text{TheTrees}[k]$ ，保存原根的孩子链后释放原根。
4. **构造临时森林：**删除根后还剩 $2^k - 1$ 个节点，所以 $\text{DeletedQueue} \rightarrow \text{CurrentSize} = (1 \ll k) - 1$ 。
5. **按递减阶数填数组：**遍历孩子链，依次填入 $k - 1, \dots, 0$ ；保存下一个兄弟后，把当前子树根的 NextSibling 清空，解除它与其他根的兄弟关系。
6. **更新规模并合并：**原队列先减去整个 B_k 的 2^k 个节点，再合入临时森林的 $2^k - 1$ 个节点，总规模恰为原来减一。[S11]

解释补充： NextSibling 断链是因为兄弟链原来表示“同一个根的孩子”，而现在这些孩子要成为独立树根。整数移位与 Infinity 哨兵还需满足实际键类型、位宽和容量约束。

4. 连续插入为何只需线性总时间 [S12–S13]

单次插入的最坏代价由最长进位链决定，整段插入序列的代价则由“创建多少棵树、连接多少次”决定。昂贵插入消去很多树，便宜插入留下新树。[S12–S13]

4.1 汇总分析 (Aggregate Analysis) [S12]

重要性：A · 必须掌握

结论是：从空队列开始连续插入 N 个键，总时间为 $O(N)$ 。[S12]

核心原理

每次插入创建一棵单节点树，每次连接使森林的树数减少一；总链接次数可由最后还剩多少棵树直接计数。

采用“一次基本插入记一单位，每次同阶连接记一单位”的代价模型：

1. N 次插入总共创建 N 棵 B_0 ，基础工作为 N 单位。

2. 每次连接把两棵树变为一棵，森林树数净减 1。设总链接次数为 L ，最后的树数为 t_N ，则 $t_N = N - L$ 。
3. 最终树数等于 N 的二进制表示中 1 的个数，记为 $\text{popcount}(N)$ 。所以 $L = N - \text{popcount}(N) \leq N - 1$ ($N \geq 1$)。
4. 总代价为 $N + L = 2N - \text{popcount}(N) < 2N$ ，从而为 $O(N)$ 。[S12]

补充推导： S12 的 Total links 在原图中留空，上面的计数补齐了汇总证明。它依赖队列从空状态开始、且每次插入只处理必要的连接。

4.2 势能分析 (Potential Method) [S13]

重要性：A · 必须掌握

取势能 Φ_i 为第 i 次插入之后森林中的树数，初始 $\Phi_0 = 0$ 。若这次插入进行了 k_i 次连接，其实际代价为 $C_i = 1 + k_i$ 。[S13]

核心原理

把森林树数作为势能；连接次数多的插入使树数减少，势能下降正好抵消这部分额外工作。

1. 新的单节点树令树数加 1，每次连接令树数减 1，所以 $\Phi_i - \Phi_{i-1} = 1 - k_i = 2 - C_i$ 。
2. 摊还代价为 $\widehat{C}_i = C_i + \Phi_i - \Phi_{i-1} = 2$ 。
3. 对 N 次插入求和，势能差相消，得到 $\sum_i C_i = 2N - \Phi_N + \Phi_0$ 。
4. 由于 $\Phi_0 = 0$ 且 $\Phi_N \geq 0$ ，实际总代价不超过 $2N$ ，因此每次插入摊还 $O(1)$ 。[S13]

“摊还常数”是对操作序列总成本的保证，不依赖随机输入；它与单次最坏 $O(\log N)$ 同时成立。这里没有证明某一次插入必然很快，也没有把合并或删除最小元改成常数时间。[S12–S13]

5. 课程项目与阅读材料 [S14–S15]

重要性：C · 了解即可

1. **最短路径项目：** 基于最小优先队列实现 Dijkstra 算法，例如使用 Fibonacci Heap；通过实现与比较判断适合该算法的数据结构。具体要求指向 Pintia，本课件未包含项目的完整规格。[S14]
2. **阅读材料：** *Data Structures and Algorithm Analysis in C*, 2nd ed., Chapter 5 (pp. 170–180) 与 Chapter 11 (pp. 430–435)；*Introduction to Algorithms*, 3rd ed., Chapter 19 (pp. 505–530)。[S15]

本讲总结

- B_k 有 2^k 个节点，根有 k 个孩子，深度 d 有 $\binom{k}{d}$ 个节点。[S2]
- 二项队列每阶最多一棵树，存在的阶数组合由规模 N 的二进制表示决定。[S3]
- 同阶树连接用较小根作父根，常数指针修改即可升一阶。[S9]
- 合并像二进制加法；删除最小元像“摘树、去根、拆成森林、再合并”。[S4、S6、S10–S11]
- 插入只沿进位链处理；最坏对数，从空队列连续 N 次插入总计线性，摊还常数。[S5、S12–S13]

复杂度与适用条件速查

操作	单次最坏时间	摊还结论	适用条件	来源
查最小元	$O(\log N)$	—	扫描所有非空根	S4
读取缓存最小元	$O(1)$	—	正确维护最小根缓存	S4
同阶树连接	$O(1)$	—	两棵树阶数相同	S9
合并	$O(\log(N_1 + N_2))$	相同量级	按阶处理并进位	S4、S10
插入	$O(\log N)$	$O(1)$	从空队列的插入序列；只处理必要进位	S5、S12–S13
删除最小元	$O(\log N)$	相同量级上界	可依次访问根的孩子子树	S6、S11
连续 N 次插入建队列	$O(N)$ 总时间	每次 $O(1)$	初始队列为空	S12–S13

存储 N 个键需要 $O(N)$ 节点空间，根数组只需覆盖 $O(\log N)$ 个有效阶；固定容量实现则按 `MaxTrees` 预留根数组。[S8]

易错点

1. 队列是森林；“每阶最多一棵”限制同阶树的数量，不限制森林总共只有一棵树。[S2–S3]
2. 规模二进制表示决定树阶数，不决定键的唯一排列。[S3]
3. `CombineTrees` 要求同阶输入；不能把任意两棵二项树简单挂接后仍称作下一阶二项树。[S9]
4. 三棵同阶树出现时，应留一棵、连两棵；不能全连成一棵二项树。[S10]
5. 删除根后的临时森林规模是 $2^k - 1$ ，从原队列摘掉的整树规模是 2^k 。[S11]
6. 摊还插入证明计算必要连接，不为额外扫描全部数组提供免费成本。[S5、S10、S12–S13]

复习自测

1. 从递归定义推出 B_k 的节点数和深度计数。（来源：[S2]）
2. $N = 13$ 时有哪些阶？为什么“唯一表示”不意味着键排列唯一？（来源：[S3]）
3. 合并某一阶有两棵或三棵树时分别怎样处理？（来源：[S4、S10]）
4. 插入到规模 $2^k - 1$ 的队列中，为何可能出现 k 次连接？（来源：[S5、S12]）
5. 为什么孩子链按阶递减，删除根时为什么要清空各新根的兄弟指针？（来源：[S9、S11]）
6. 用树数计数和势能法各证明一次 N 次插入为 $O(N)$ 。（来源：[S12–S13]）

术语速查

术语	概念	来源
二项树 (Binomial Tree)	两棵同阶树递归连接得到的树	S2
二项队列 (Binomial Queue)	满足堆序、每阶至多一棵二项树的森林	S2–S3
二项系数 (Binomial Coefficient)	$\binom{k}{d}$ ，表示 B_k 深度 d 的节点数	S2
进位 (Carry)	两棵本阶树连接后产生的下一阶树	S4、S10
孩子—兄弟表示 (Child–Sibling Representation)	用首孩子和下一兄弟指针表示多孩子树	S8–S9

术语	概念	来源
汇总分析 (Aggregate Analysis)	对一段操作的实际总成本直接计数	S12
势能法 (Potential Method)	用状态势能变化修正单次实际代价	S13

待核对

- [待核对 S2] 对教材 Theorem 5.1 的提问未附完整定理和输入条件；若要比普通二叉堆“平均插入时间”，需核对对应定理，不能把线性建堆与任意逐次插入的最坏成本混为一谈。
- [待核对 S10–S11] 代码节选未给出全部队列对象的重置与释放流程，正式实现需核对 Merge 消耗第二输入后的 CurrentSize 和临时对象管理。
- [待核对 S14] Pintia 上的项目规格不在本课件中；需要课程当前项目说明才能确认实现、测试与提交要求。